LITERATURE REVIEW • DIRECT SCIENCE FOUNDATION

PEER-REVIEWED FOUNDATIONS

No Analogies • Literal Technical Definitions

Why the Triad Makes AI Safe and Deterministic

A Direct Literature Review of Knowledge Graphs, Finite State Machines, and Context Graphs

1 The Problem: Large Language Models Are Probabilistic Token Predictors

A Large Language Model is a neural network. It calculates numbers called probabilities. When given input text, it calculates which word token has the highest probability of appearing next.

Because it picks word tokens based on probabilities, its output is non-deterministic. Non-deterministic means running the exact same input twice can give two different outputs. Sometimes the model predicts a word token that is false. In computer science, this is called a hallucination. When an autonomous software program uses an unconstrained Large Language Model to make decisions, it can skip required verification steps, call incorrect application programming interface (API) tools, or violate legal rules.

The Research Question: How do computer scientists make an autonomous system deterministic when its language engine is probabilistic?

2 The Core Theorem: The Triad Architecture

To make an AI system deterministic, computer science does not rely on the language model to police itself. Instead, the language model is enclosed inside three mathematical structures:

$$\text{Deterministic Outcome} = F_{\text{Rules}}(\ \text{Live Situation} \mid \text{World Model}\ )$$

This theorem states that an outcome is deterministic when an action is computed by an operational transition function (F_{Rules}), taking the current factual state (`Live Situation`), and verifying it against formal constraints (`World Model`).

3 Pillar 1: World Model (Knowledge Graph Literature)

Definition: A Knowledge Graph is a data structure made of nodes and directed labeled edges. A node represents an entity or a rule. A directed edge represents the exact semantic relationship between two nodes.

> **Gary Marcus (2020) — *The Next Decades in AI: Four Steps Towards Robust AI***
> Marcus proves that statistical pattern recognition cannot understand facts or rules. He shows that robust artificial intelligence requires an explicit symbolic world model that defines invariant rules. An invariant rule is a rule that must never be broken under any condition.

> **Aidan Hogan et al. (2021) — *Knowledge Graphs* (ACM Computing Surveys)**
> Hogan and co-authors provide the formal graph schema for knowledge representation. They prove that knowledge graphs store declarative rules using formal ontologies. A graph engine can execute graph queries in polynomial time to verify whether a policy condition is mathematically true or false.

> **Judea Pearl (2009) — *Causality: Models, Reasoning, and Inference***
> Pearl proves that observing patterns does not tell a computer what will happen when an action is taken. A system requires a directed graph of causal dependencies to compute valid outcomes.

4 Pillar 2: Operational Rules (Finite State Machine Literature)

Definition: A Finite State Machine is an automaton defined by a finite set of states, a set of input symbols, and a deterministic transition function. It can only be in one state at a time. It can only move to a new state if a transition edge exists and its guard conditions evaluate to true.

P. J. Ramadge and W. M. Wonham (1987) — *Supervisory Control Theory* (SIAM Journal)
Ramadge and Wonham proved a fundamental mathematical theorem: if you have an unconstrained, unpredictable actor (called the plant), you can place a supervisor automaton (a Finite State Machine) around it. The supervisor disables any event that leads to an illegal state. This mathematically guarantees that the system never enters an unauthorized state.

David Harel (1987) — *Statecharts: A Visual Formalism for Complex Systems*
Harel created the formal mathematics for states with guards and actions. A guard is a boolean expression that must equal true before a state transition can occur. If the guard equals false, the machine remains in its current state.

Mohammed Alshiekh et al. (2018) — *Safe Reinforcement Learning via Shielding* (AAAI)
Alshiekh and co-authors created the formal "shield". When an AI algorithm outputs an action, the shield checks the action against safety specifications. If the action violates a specification, the shield blocks the action and outputs an error. In software engineering, this is HTTP 403 Forbidden.

5 Pillar 3: Live Situation (Context Graph & Provenance Literature)

Definition: A Context Graph is an instance graph that records the active session state, verified parameter values, and cryptographic references to source evidence files.

Jon Barwise and John Perry (1983) — *Situations and Attitudes* (MIT Press)
Barwise and Perry created Situation Semantics. They proved that linguistic statements cannot be verified in the abstract. A statement can only be evaluated for truth against a specific, bounded situation containing specific objects and spatio-temporal facts.

Luc Moreau et al. (2013) — *W3C PROV-DM: The PROV Data Model*
The World Wide Web Consortium (W3C) formalized data provenance. Provenance means the history of an entity's origin. PROV-DM defines directed relationships showing which process generated an entity and which document was used to derive a parameter value. This prevents an agent from using ungrounded numbers.

6 The Synthesis: Neuro-Symbolic Verification

Artur d'Avila Garcez and Luis C. Lamb (2023) — *Neurosymbolic AI: The 3rd Wave*
Garcez and Lamb show that statistical neural models and symbolic logic engines perform two different functions. Neural models translate unstructured human text into candidate parameters. Symbolic engines verify those parameters against formal logic. Combining them creates an architecture where perception is neural, but execution is symbolic and provable.

Subbarao Kambhampati et al. (2024) — *LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks* (CACM)
Kambhampati and his team proved through empirical testing that Large Language Models cannot independently guarantee the correctness of multi-step plans. However, when an external model verifier audits each step against formal state rules, the system achieves provable correctness.

FINAL SCIENTIFIC CONCLUSION

A Large Language Model by itself cannot be deterministic because it computes word probabilities. Determinism is achieved by delegating execution control to a **Finite State Machine**, binding policies to a **Knowledge Graph**, and grounding data in a **Context Graph**. Together, these three mathematical structures make the system 100% deterministic, auditable, and safe.